



ASSURE DEFI[®]
THE VERIFICATION GOLD STANDARD

SECURITY ASSESSMENT REPORT



NAME:

TIPCRYP

STATUS:

FAIL

DATE:

10/06/2026



Risk Analysis

Vulnerability summary

Classification	Description
 High	Vulnerabilities that can lead to loss or theft of funds, permanent locking of assets, unauthorized minting/burning, protocol insolvency, or full control of the contract or protocol. These issues are typically exploitable on-chain and require immediate remediation.
 Medium	Vulnerabilities that weaken protocol security or trust assumptions but do not directly enable immediate loss of funds under normal conditions. Exploitation may require specific conditions, privileged roles, or external failures.
 Low	Issues that have limited direct impact on funds or protocol integrity but may cause unexpected behavior, reduced resilience, or future exploitability if combined with other weaknesses.
 Informational	Findings that do not represent security vulnerabilities, but highlight code quality, clarity, maintainability, or best-practice improvements that may enhance long-term safety and auditability.

SCOPE

Target Code And Revision

Project	Assure
Codebase	https://basescan.org/address/0x0Fb810528d4213785d80588A9aa47664a6396862#code
Audit Methodology	Static, Manual

Detailed Technical Report



1. Token allocation formula is 10x too generous

Issue:

Prices are documented as cents per token, for example `stagePrice[1] = 5` means \$0.05 per token. Stablecoin amounts are converted to cents using:

```
uint256 usdCents = amt / STABLE_SCALE; // STABLE_SCALE = 1e4
```

That part is correct for 6-decimal USDC/USDT: $25_000_000 / 1e4 = 2500$ cents = \$25.

However, `_buy()` calculates token allocation as:

```
uint256 base = (usdCents * 1e19) / stagePrice[currentStage];
```

It should be:

```
uint256 base = (usdCents * 1e18) / stagePrice[currentStage];
```

The extra factor of 10 means every buyer receives 10x the intended number of TCM tokens. The same incorrect formula is also used in the ETH `minTokensOut` preview.

The price comments, stage prices, `_buy()` calculation, and `investETH()` preview are visible in the uploaded source.

Remediation:

Replace all token allocation formulas using 1e19 with 1e18.

uint256 base = (usdCents * 1e18) / stagePrice[currentStage];

Do the same in investETH() preview logic, and preferably centralize the calculation:

```
function _quoteTokens(uint256 usdCents, uint8 stage) internal view returns (uint256) {  
    uint256 base = (usdCents * 1e18) / stagePrice[stage];  
    return base + (base * stageBonus[stage]) / 100;  
}
```

Add unit tests for \$25, \$15,000, and each stage price.



2. owner can bypass treasury and withdraw USDT/USDC via recoverERC20

Issue:

The contract has dedicated withdrawal functions for USDT and USDC that require:

require(msg.sender == treasury, "TREASURY_ONLY");

But recoverERC20() is callable by owner and allows recovery of any ERC20 except TCM when totalAllocated != 0:

```
function recoverERC20(address token, uint256 amount) external onlyOwner {  
    require(token != address(0), "ZERO_ADDR");  
    require(token != address(tcmToken) || totalAllocated == 0, "TCM_ALLOCATED");  
    IERC20(token).safeTransfer(owner, amount);  
}
```

This means the owner can withdraw usdtToken and usdcToken balances directly to owner, bypassing the treasury role entirely. The treasury-only withdrawal functions and owner recovery function are both present in the source.

Remediation:

Exclude payment tokens from generic recovery:

```
require(token != address(usdtToken), "NO_RECOVER_USDT");
```

```
require(token != address(usdcToken), "NO_RECOVER_USDC");
```

Or route recovered payment tokens to treasury and require msg.sender == treasury.

Recommended safer pattern:

```
function recoverERC20(address token, uint256 amount) external onlyOwner {  
    require(token != address(0), "ZERO_ADDR");  
    require(token != address(tcmToken), "NO_TCM_RECOVERY");  
    require(token != address(usdtToken), "NO_USDT_RECOVERY");  
    require(token != address(usdcToken), "NO_USDC_RECOVERY");  
    IERC20(token).safeTransfer(owner, amount);  
}
```

3. Excess/unsold TCM can become permanently unrecoverable

Issue:

totalAllocated is increased on every purchase:

totalAllocated += total;

But it is never decreased when users claim. In claim(), the user's allocation is zeroed, but totalAllocated remains unchanged:

uint256 amt = allocation[msg.sender];

claimed[msg.sender] = true;

allocation[msg.sender] = 0;

tcmToken.safeTransfer(msg.sender, amt);

Then recoverERC20() blocks TCM recovery forever as long as totalAllocated != 0:

require(token != address(tcmToken) || totalAllocated == 0, "TCM_ALLOCATED");

So after the first purchase, totalAllocated becomes permanently nonzero. Even after all buyers claim and there are no remaining user allocations, excess TCM cannot be recovered. The relevant claim and recovery logic is shown in the source.

Remediation:

Separate cumulative sold amount from outstanding claim liability.

Example:

uint256 public totalSold;

uint256 public totalUnclaimed;

On purchase:

totalSold += total;

totalUnclaimed += total;

On claim:

totalUnclaimed -= amt;

Then allow recovery only of excess TCM:

```
function recoverExcessTCM(uint256 amount) external onlyOwner {  
    uint256 balance = tcmToken.balanceOf(address(this));  
    require(balance >= totalUnclaimed + amount, "ALLOCATED_TCM");  
    tcmToken.safeTransfer(owner, amount);  
}
```

4. ETH oracle accepts very stale prices and lacks L2 sequencer checks

Issue:

`_ethPrice()` accepts oracle data as long as it is less than 90,000 seconds old:

`require(block.timestamp - updatedAt < 90000, "STALE_PRICE");`

That is roughly 25 hours. For a public ETH-denominated token sale, this is a large stale-price window. If the stale ETH/USD price is higher than the real market price, buyers can acquire TCM at a discount. The code comments also mention Base/Polygon assumptions elsewhere; if deployed on an L2 such as Base, ETH purchases should also check sequencer uptime before trusting the feed. The current oracle validation only checks positive answer, timestamp, stale age, and answered round.

Remediation:

Use a feed-specific heartbeat much shorter than 90,000 seconds.

```
uint256 public constant MAX_ORACLE_DELAY = 1 hours;
```

```
require(block.timestamp - updatedAt <= MAX_ORACLE_DELAY, "STALE_PRICE");
```

For L2 deployments, add a sequencer uptime feed check with a grace period before accepting ETH purchases.

Also consider:

```
require(answeredInRound == roundId, "INCOMPLETE_ROUND");
```

and an owner-controlled emergency pause specifically for ETH purchases.





MEDIUM

1. minTokensOut can be bypassed across stage expiry; stablecoin buys have no slippage guard

Issue:

investETH() checks minTokensOut before _buy() advances expired stages:

```
if (minTokensOut > 0) {  
    uint256 base = (usdCents * 1e19) / stagePrice[currentStage];  
    uint256 preview = base + (base * stageBonus[currentStage]) / 100;  
    require(preview >= minTokensOut, "SLIPPAGE");  
}
```

_buy(msg.sender, usdCents);

But _buy() then calls:

_advanceExpiredStages();

So the slippage check can be performed against an old stage, while the actual purchase executes in a later, more expensive stage. The ETH preview and _buy() stage advancement are shown in the source.

USDT and USDC purchases have no minTokensOut parameter at all.

Remediation:

Advance stages before quote calculation:

`_advanceExpiredStages();`

Then compute the quote once using a shared helper and enforce the final result.

Also add slippage protection to stablecoin buys:

`function investUSDT(uint256 amt, uint256 minTokensOut) external nonReentrant`

`function investUSDC(uint256 amt, uint256 minTokensOut) external nonReentrant`

Add a deadline parameter to prevent stale signed/user-submitted transactions from executing later than expected.



2. Users can create unclaimable allocations if buying after claiming

Issue:

claim() permanently sets:

claimed[msg.sender] = true;

allocation[msg.sender] = 0;

But _buy() does not prevent a user from buying after they have claimed. If a user buys again after claiming, allocation[user] increases, but future claim() calls revert because:

require(!claimed[msg.sender], "ALREADY_CLAIMED");

The claim logic and buy allocation updates are both present in the source.

Remediation:

Prevent purchases once claims are live or once TGE has arrived:

```
require(!claimEnabled, "CLAIMS_STARTED");
```

```
require(block.timestamp < tgeTimestamp, "SALE_ENDED");
```

Alternatively, remove the permanent claimed boolean and use amount-based claiming, where users can claim any positive allocation.





No low severity issues were found.





INFORMATIONAL

No informational severity issues were found.



Technical Findings Summary

Findings

Vulnerability Level	Total	Pending	Not Apply	Acknowledged	Partially Fixed	Fixed
● High	4					
● Medium	2					
● Low						
● Informational						

Assessment Results

Score Results

Review	Score
Global Score	65/100
Assure KYC	Not completed
Audit Score	65/100

The Following Score System Has been Added to this page to help understand the value of the audit, the maximum score is 100, however to attain that value the project must pass and provide all the data needed for the assessment. Our Passing Score has been changed to 84 Points for a higher standard, if a project does not attain 85% is an automatic failure. Read our notes and final assessment below. The Global Score is a combination of the evaluations obtained between having or not having KYC and the type of contract audited together with its manual audit.

Audit FAIL

Following our comprehensive security audit of the token contract for the **TipCryp** project, the project did not fulfill the necessary criteria required to pass the security audit.



Disclaimer

Assure Defi has conducted an independent security assessment to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the reviewed code for the scope of this assessment. This report does not constitute agreement, acceptance, or advocating for the Project, and users relying on this report should not consider this as having any merit for financial Token in any shape, form, or nature. The contracts audited do not account for any economic developments that the Project in question may pursue, and the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude, and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are entirely free of exploits, bugs, vulnerabilities or deprecation of technologies. All information provided in this report does not constitute financial or investment in Token, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence, regardless of the findings presented. Information is provided 'as is, and Assure Defi is under no covenant to audit completeness, accuracy, or solidity of the contracts. In no event will Assure Defi or its partners, employees, agents, or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions or actions with regards to the information provided in this audit report. The assessment of Tokens provided by Assure Defi are subject to dependencies and are under continuing development. You agree that your access or use, including but not limited to any Tokens, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies with high levels of technical risk and uncertainty. The assessment reports could include false positives, negatives, and unpredictable results. The Token may access, and depend upon, multiple layers of third parties.

